

Study Report: Data Engineer - Career Path & Core Skills

Written by Gagan Pasupuleti

Family:	Data Engineer
Level:	Beginner to Advanced
Tracks:	Beginner, Intermediate, Advanced
Chapters:	18
Source:	Official docs, free libraries, and industry-standard references (see sources and books)

Official sources and free libraries

- <https://spark.apache.org/docs/latest/>
- <https://airflow.apache.org/docs/>
- <https://docs.getdbt.com/>
- <https://docs.snowflake.com/>
- <https://cloud.google.com/bigquery/docs>
- <https://greatexpectations.io/docs/>

Summary

This report maps the Data Engineer role for 2024-2026: ETL/ELT pipelines, SQL, Python, Apache Spark, Airflow, dbt, cloud warehouses (Snowflake, BigQuery, Databricks), and data quality. Covers batch and streaming, table formats (Delta/Iceberg), observability, and cost-aware design.

Skill stack by level

Beginner

- ETL vs ELT concepts
- SQL for transformations
- Python scripting for pipelines
- Parquet/CSV/JSON file formats
- Cloud warehouse basics
- Git and code review
- Idempotent job design

Intermediate

- PySpark DataFrame API (3.5+)
- Airflow DAGs and operators
- dbt models, tests, and docs
- Great Expectations suites
- CDC and incremental loads

- Delta Lake / Apache Iceberg
- Terraform for data infra
- Docker for local pipeline dev

Advanced

- Kafka + Structured Streaming
- Data mesh and data contracts
- Multi-cloud federation patterns
- Pipeline observability (OpenLineage)
- FinOps for warehouse spend
- Real-time feature pipelines
- Zero-downtime schema evolution
- Disaster recovery for data platforms

Recommended books

1. Fundamentals of Data Engineering - Joe Reis & Matt Housley

Level: Beginner | Modern data stack, pipelines, and team roles in 2024+.

2. Designing Data-Intensive Applications - Martin Kleppmann

Level: Advanced | Storage, replication, streaming, and distributed systems.

3. Learning Spark - Jules Damji et al.

Level: Intermediate | PySpark DataFrames, SQL, and cluster processing at scale.

4. dbt Developer Hub - dbt Labs

Level: Intermediate | Analytics engineering: models, tests, and documentation in SQL.

5. The Data Warehouse Toolkit - Ralph Kimball

Level: Intermediate | Dimensional modeling, star schema, and ETL design.

End-to-end projects

Project 1: CSV-to-Warehouse Batch Pipeline

Beginner | 2 weeks - Python ETL script: extract CSV, transform with pandas, load to PostgreSQL with logging.

- Extract: Raw landing table
- Transform: Staging table
- Load: Final table populated
- Schedule: Scheduled job running

Project 2: Airflow + dbt Analytics Pipeline

Intermediate | 4-5 weeks - Airflow DAG orchestrates dbt models on BigQuery/Snowflake with data quality tests.

- dbt: dbt docs site
- Airflow: DAG running daily
- Quality: Quality report
- Alert: Alert screenshot

Project 3: Real-Time Streaming Pipeline

Advanced | 6-8 weeks - Kafka ? Spark Structured Streaming ? Delta Lake with monitoring and exactly-once semantics.

- Kafka: Topic flowing

- Spark: Streaming job running
- Delta: Delta table
- Monitor: Grafana dashboard

Chapters

Track: Beginner

Chapter 1: Data Engineer Role Overview [Beginner]

Chapter focus: Data Engineer Role Overview** *(Level: Beginner)

Data engineers build reliable pipelines that move and transform data at scale. You own ingestion, modeling layers, orchestration, and data quality - enabling analysts and ML teams. Cloud-native tooling (managed Spark, serverless warehouses) dominates in 2024-2026.

Code Reference:

```
# Pipeline layers: ingest -> bronze -> silver -> gold -> serve
```

What it shows: Layered architecture separates raw from trusted data.

Topic guide

What it actually is

A data engineer designs and operates data pipelines and platform infrastructure.

When to use it

When data volume, variety, or SLA exceeds spreadsheet/manual scripts.

Where to use it

SaaS analytics, fintech ledgers, ad-tech events, and ML feature stores.

Real use example

CodeQuest nightly exports land in S3; your job makes them queryable by 6 AM for teachers.

Chapter 2: ETL vs ELT and Batch vs Streaming [Beginner]

Chapter focus: ETL vs ELT and Batch vs Streaming** *(Level: Beginner)

ETL transforms before load; ELT loads raw then transforms in the warehouse (dbt). Batch runs on schedule; streaming processes events continuously. Pick batch unless sub-hour latency is a hard requirement - streaming adds operational complexity.

Code Reference:

```
batch_schedule = '0 2 * * *' # daily 2 AM UTC
# streaming: micro-batch or continuous with watermarks
```

What it shows: Cron illustrates batch; streaming needs separate ops playbook.

Topic guide

What it actually is

ETL/ELT define where transformation runs; batch/stream define timing.

When to use it

Designing any new data source integration.

Where to use it

Nightly ERP syncs vs clickstream real-time dashboards.

Real use example

Finance needs T+1 batch; fraud team needs 30-second streaming - you run both patterns.

Chapter 3: SQL for Data Transformations [Beginner]

Chapter focus: SQL for Data Transformations** *(Level: Beginner)

Engineers write heavy SQL: MERGE for upserts, window functions for dedupe, CREATE TABLE AS for snapshots. Use explicit column lists; avoid SELECT *. Version SQL in Git with migration tools.

Code Reference:

```
MERGE INTO curated.orders t
USING staging.orders s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET amount = s.amount
WHEN NOT MATCHED THEN INSERT (id, amount) VALUES (s.id, s.amount);
```

What it shows: MERGE idempotently applies daily deltas without duplicate rows.

Topic guide

What it actually is

SQL remains the transformation lingua franca in warehouses.

When to use it

Curated layer builds, incremental syncs, and backfills.

Where to use it

Snowflake, BigQuery, Redshift, and Databricks SQL.

Real use example

Staging duplicate keys are resolved with ROW_NUMBER before MERGE - finance totals stay correct.

Chapter 4: Python for Pipeline Scripts [Beginner]

Chapter focus: Python for Pipeline Scripts** *(Level: Beginner)

Python orchestrates file I/O, API pagination, and cloud SDK calls. Structure projects with click or typer CLIs, pydantic for config, and logging not print. Pin dependencies in requirements.lock.

Code Reference:

```
import logging
import pandas as pd

logging.basicConfig(level=logging.INFO)

def load_csv(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    logging.info('Loaded %s rows from %s', len(df), path)
    return df
```

What it shows: Structured logging aids production debugging in Airflow task logs.

Topic guide**What it actually is**

Python glues together extraction and load steps warehouses cannot do natively.

When to use it

API ingestion, file validation, and custom transforms.

Where to use it

S3?warehouse loaders, SaaS extractors, and notification hooks.

Real use example

A HubSpot extractor paginates contacts and lands parquet partitioned by extract_date.

Chapter 5: File Formats: Parquet, JSON, Avro [Beginner]**Chapter focus: File Formats: Parquet, JSON, Avro***(Level: Beginner)**

Parquet is columnar - efficient for analytics. JSON lines suit semi-structured logs. Avro with schema registry protects streaming contracts. Store raw immutable; transform to Parquet in silver.

Code Reference:

```
df.to_parquet('s3://bucket/silver/orders/date=2025-07-01/data.parquet', index=False)
```

What it shows: Partition columns (date=) enable predicate pushdown - cheaper queries.

Topic guide**What it actually is**

File format choice affects cost, schema evolution, and query speed.

When to use it

Lakehouse storage and exchange between systems.

Where to use it

Data lakes, event archives, and ML training sets.

Real use example

Switching CSV exports to Snappy Parquet cuts storage 80% and speeds Spark reads 5x.

Chapter 6: Cloud Warehouse Basics [Beginner]

Chapter focus: Cloud Warehouse Basics** *(Level: Beginner)

Snowflake separates storage/compute; BigQuery is serverless; both support nested types and RBAC. Understand warehouses/virtual warehouses, roles, and cost monitors before production loads.

Code Reference:

```
CREATE WAREHOUSE etl_wh WITH WAREHOUSE_SIZE = 'MEDIUM' AUTO_SUSPEND = 60;
```

What it shows: Auto-suspend stops idle compute billing - critical FinOps habit.

Topic guide

What it actually is

Cloud warehouses are managed analytical databases at petabyte scale.

When to use it

Central curated layer for BI and ML features.

Where to use it

Enterprise analytics, ad-hoc SQL, and dbt transforms.

Real use example

You right-size warehouse from XL to S after profiling - monthly bill drops 40%.

Track: Intermediate

Chapter 7: PySpark DataFrames (Spark 3.5+) [Intermediate]

Chapter focus: PySpark DataFrames (Spark 3.5+)** *(Level: Intermediate)

PySpark distributes transforms across clusters. Use DataFrame API (not RDDs); lazy evaluation builds logical plans optimized by Catalyst. Cache judiciously - memory is finite.

Code Reference:

```
from pyspark.sql import functions as F

df = spark.read.parquet('s3://bucket/events/')
agg = df.groupBy('country').agg(F.count('*').alias('events'))
agg.write.mode('overwrite').parquet('s3://bucket/gold/events_by_country/')
```

What it shows: groupBy + write parquet is the canonical silver?gold pattern.

Topic guide

What it actually is

PySpark processes terabyte-scale data on commodity clusters.

When to use it

Heavy transforms that choke single-node pandas.

Where to use it

Lakehouse ETL, log aggregation, and ML preprocessing.

Real use example

Daily 500GB clickstream dedupe finishes in 12 minutes on 8 workers.

Chapter 8: Apache Airflow DAGs [Intermediate]**Chapter focus: Apache Airflow DAGs** *(Level: Intermediate)**

Airflow DAGs define task dependencies with operators (Python, SQL, S3). Use idempotent tasks; retry with exponential backoff; store secrets in Connections/Variables backend (Vault, AWS SM).

Code Reference:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG('daily_orders', start_date=datetime(2025,1,1), schedule='0 2 * * *') as dag:
    extract = PythonOperator(task_id='extract', python_callable=extract_orders)
    load = PythonOperator(task_id='load', python_callable=load_to_warehouse)
    extract >> load
```

What it shows: >> sets dependency - load waits for successful extract.

Topic guide**What it actually is**

Airflow orchestrates when and in what order pipeline tasks run.

When to use it

Any multi-step pipeline needing schedules, retries, and observability.

Where to use it

dbt + Spark + quality checks coordinated nightly.

Real use example

Failed load triggers Slack alert; retry succeeds after upstream API blip.

Chapter 9: dbt Models, Tests, and Documentation [Intermediate]**Chapter focus: dbt Models, Tests, and Documentation** *(Level: Intermediate)**

dbt compiles SQL models with ref() dependencies. Add tests: unique, not_null, relationships.

Generate docs site for lineage. Use tags and selectors for CI slim runs.

Code Reference:

```
-- models/fct_orders.sql
select
  order_id,
  customer_id,
  amount
from {{ ref('stg_orders') }}
where status = 'completed'
```

What it shows: ref() builds DAG of models - runs in correct order automatically.

Topic guide

What it actually is

dbt is analytics engineering: transform in warehouse with software practices.

When to use it

Gold layer metrics consumed by BI and analysts.

Where to use it

Snowflake/BigQuery + GitHub Actions CI.

Real use example

relationships test catches orphan customer_id before executives see wrong LTV.

Chapter 10: Data Quality with Great Expectations [Intermediate]

Chapter focus: Data Quality with Great Expectations** *(Level: Intermediate)

Define Expectations on columns: expect_column_values_to_be_between, expect_unique. Run validation checkpoints in Airflow; fail pipeline on critical violations; quarantine bad batches.

Code Reference:

```
import great_expectations as gx
context = gx.get_context()
suite = context.add_expectation_suite('orders_suite')
# expect_column_values_to_not_be_null on order_id
```

What it shows: Checkpoints integrate with pipeline gates - bad data never reaches gold.

Topic guide

What it actually is

Data quality frameworks codify trust rules executable in CI/CD.

When to use it

After ingest and before publishing to consumers.

Where to use it

Finance pipelines, healthcare claims, and public metrics.

Real use example

Null order_id batch routed to quarantine bucket - analysts notified with GE Data Docs link.

Chapter 11: CDC and Incremental Loading [Intermediate]

Chapter focus: CDC and Incremental Loading** *(Level: Intermediate)

Change Data Capture tracks inserts/updates/deletes. Patterns: timestamp watermark, log-based Debezium, merge keys. Handle late-arriving facts with lookback windows.

Code Reference:

```
SELECT * FROM source.orders
WHERE updated_at >= '{{ prev_run_ts }}'
  AND updated_at < '{{ curr_run_ts }}'
```

What it shows: Watermark bounds incremental pull - idempotent with MERGE on id.

Topic guide

What it actually is

Incremental loads minimize data transfer and warehouse spend.

When to use it

Large tables where full reloads are too slow or expensive.

Where to use it

CRM, billing, and inventory sources with frequent updates.

Real use example

7-day lookback catches late updates without nightly full table scan.

Chapter 12: Delta Lake and Apache Iceberg [Intermediate]

Chapter focus: Delta Lake and Apache Iceberg** *(Level: Intermediate)

Open table formats add ACID, time travel, and schema evolution on object storage. Delta is Databricks-native; Iceberg is engine-neutral (Spark, Flink, Trino). Pick based on stack.

Code Reference:

```
spark.sql("""
MERGE INTO silver.orders t
USING bronze.orders_updates s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")
```

What it shows: MERGE on Delta applies CDC batches atomically.

Topic guide

What it actually is

Lakehouse table formats bring warehouse reliability to data lakes.

When to use it

Medallion architectures on S3/ADLS.

Where to use it

Unified batch + streaming storage layers.

Real use example

Time travel recovers yesterday's table version after bad deploy - no full restore.

Chapter 13: Infrastructure as Code for Data [Intermediate]

Chapter focus: Infrastructure as Code for Data *(Level: Intermediate)**

Terraform modules provision buckets, IAM roles, warehouses, and Airflow environments. Peer review infra changes; separate state per env; never hardcode secrets.

Code Reference:

```
resource "snowflake_warehouse" "etl" {  
  name           = "ETL_WH"  
  warehouse_size = "MEDIUM"  
  auto_suspend   = 60  
}
```

What it shows: Terraform codifies warehouse sizing - reproducible across dev/prod.

Topic guide**What it actually is**

IaC makes data platform changes auditable and repeatable.

When to use it

Bootstrapping new environments or disaster recovery.

Where to use it

Multi-env analytics platforms on AWS/Azure/GCP.

Real use example

New region stack applied from module in 30 minutes vs manual console clicks.

Track: Advanced

Chapter 14: Kafka and Spark Structured Streaming [Advanced]

Chapter focus: Kafka and Spark Structured Streaming *(Level: Advanced)**

Structured Streaming reads Kafka topics with subscribe option; watermarks bound late data;

checkpoint locations enable exactly-once-ish sinks. Monitor consumer lag.

Code Reference:

```
df = (spark.readStream.format('kafka')
      .option('subscribe', 'clickstream')
      .load())
query = df.writeStream.format('delta').option('checkpointLocation', 's3://chk/').start()
```

What it shows: checkpointLocation stores offsets - restart without duplicating sinks.

Topic guide

What it actually is

Stream processing handles continuous event pipelines.

When to use it

When sub-minute latency is required for ops or fraud use cases.

Where to use it

Ad-tech, gaming telemetry, and IoT.

Real use example

Consumer lag alert fires at 10k messages - autoscaler adds Spark executors.

Chapter 15: Data Mesh and Data Contracts [Advanced]

Chapter focus: Data Mesh and Data Contracts** *(Level: Advanced)

Data mesh decentralizes ownership to domains with federated governance. Data contracts define schema, SLAs, and quality between producers/consumers - enforced in CI.

Code Reference:

```
contract:
  name: orders_v2
  schema: { order_id: uuid, amount: decimal }
  sla: { freshness_minutes: 60 }
```

What it shows: YAML contracts versioned in Git gate breaking schema changes.

Topic guide

What it actually is

Data mesh + contracts scale ownership beyond central platform team.

When to use it

When central data teams become bottlenecks for domain teams.

Where to use it

Large enterprises adopting product-thinking for data.

Real use example

Checkout domain publishes orders_v2 contract - downstream ML features pin version.

Chapter 16: Pipeline Observability with OpenLineage [Advanced]

Chapter focus: Pipeline Observability with OpenLineage *(Level: Advanced)**

Emit lineage events from Airflow, Spark, and dbt to Marquez/DataHub. Correlate failures to downstream dashboards. SLI: freshness, completeness, error rate.

Code Reference:

```
from openlineage.client import OpenLineageClient
# emit START/COMPLETE events per task with run_id
```

What it shows: Lineage links Airflow task failures to impacted BI dashboards.

Topic guide

What it actually is

Observability makes data platforms debuggable like microservices.

When to use it

When pipeline failures impact multiple downstream dashboards and teams.

Where to use it

Data platform SRE and incident response.

Real use example

PagerDuty shows which gold table stale - root cause traced to upstream API timeout.

Chapter 17: FinOps for Warehouse Spend [Advanced]

Chapter focus: FinOps for Warehouse Spend *(Level: Advanced)**

Tag workloads by team; use auto-suspend; right-size warehouses; partition pruning; avoid SELECT * in scheduled jobs. Review query history weekly - top 10 queries often reveal bugs.

Code Reference:

```
SELECT warehouse_name, SUM(credits_used)
FROM snowflake.account_usage.warehouse_metering_history
GROUP BY 1 ORDER BY 2 DESC;
```

What it shows: Metering history identifies runaway warehouses.

Topic guide

What it actually is

FinOps controls cloud analytics cost without blocking teams.

When to use it

Rising warehouse bills or capacity planning.

Where to use it

Snowflake, BigQuery, and Databricks billing reviews.

Real use example

Moving full-table scan to incremental dbt model saves \$8k/month.

Chapter 18: Disaster Recovery for Data Platforms [Advanced]**Chapter focus: Disaster Recovery for Data Platforms** *(Level: Advanced)**

Document RPO/RTO per dataset. Replicate buckets cross-region; export warehouse backups; test restore quarterly. Runbooks for total region outage include read-only degraded mode.

Code Reference:

```
# DR drill checklist
# 1. Restore bucket prefix from replica
# 2. Replay Airflow from last checkpoint
# 3. Validate row counts vs source
```

What it shows: Quarterly drills prove backups are not vanity.

Topic guide**What it actually is**

DR planning keeps analytics alive through outages.

When to use it

Business-critical revenue and compliance datasets.

Where to use it

Banks, healthcare, and global SaaS.

Real use example

Region outage: failover warehouse in EU reads replicated Iceberg - dashboards up in 2 hours.